



Command

Instituto Federal do Mato Grosso (IFMT)
Pontes e Lacerda - Fronteira Oeste



Conteúdo da aula

- **Introdução**
- Classes em Python
- Introdução a Padrões de Projeto
 - Significado
 - Catálogo
- Conclusão
- Exercícios propostos
- Referências bibliográficas

Introdução

- **Categorização dos padrões de projeto**

Criação	Estrutural	Comportamental	
Factory Method	Adapter	Interpreter	Observer
Abstract Factory	Proxy	Template Method	State
Builder	Bridge	Chains of Respon.	Strategy
Prototype	Composite	Command	Visitor
Singleton	Decorator	Iterator	Memento
	Façade	Mediator	
	Flyweight		



Conteúdo da aula

- **Introdução**
- Classes em Python
- Introdução a Padrões de Projeto
 - Significado
 - Catálogo
- Conclusão
- Exercícios propostos
- Referências bibliográficas

Revisão de Python





Conteúdo da aula

- Introdução
- Classes em Python
- Introdução a Padrões de Projeto
 - Significado
 - Catálogo
- Conclusão
- Exercícios propostos
- Referências bibliográficas

Command



- **Intenção**: Encapsular uma solicitação como um objeto, desta forma permitindo parametrizar clientes com diferentes solicitações, enfileirar ou fazer registro (log) de solicitações e suportar operações que podem ser desfeitas
 - Uma solicitação é encapsulada dentro de um objeto
 - É possível desfazer a execução da solicitação
 - Separa transmissor e receptor
 - Possibilidade de criar uma lista de solicitações

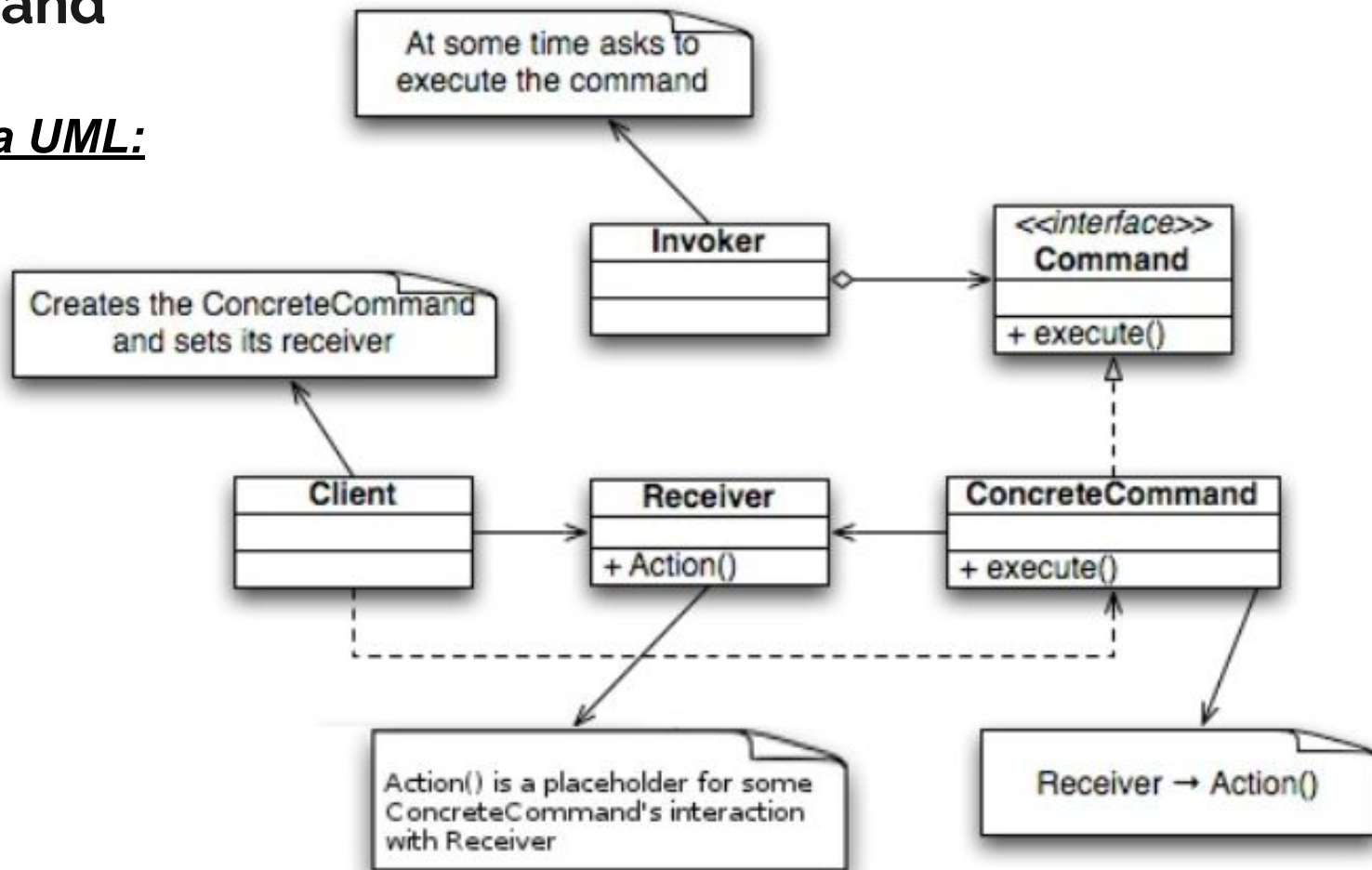
Command

- **Problema**: Criar um gerenciador de tarefas, no qual cada tarefa pode ser uma ação diferente realizada no sistema. É necessário que as tarefas sejam priorizadas e que possam ser desfeitas.
 - **Solução**: Usar o padrão de projeto Command



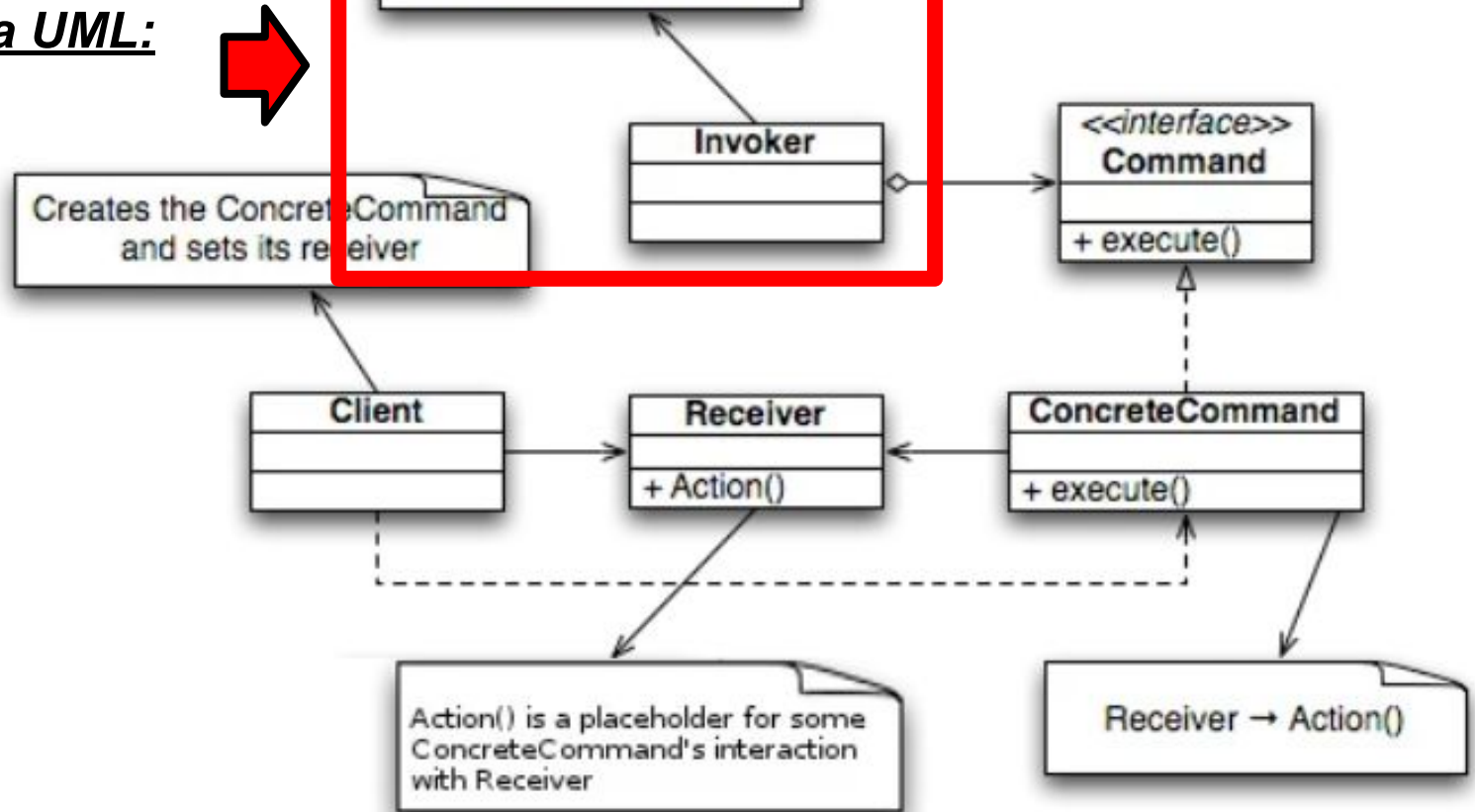
Command

- Diagrama UML:



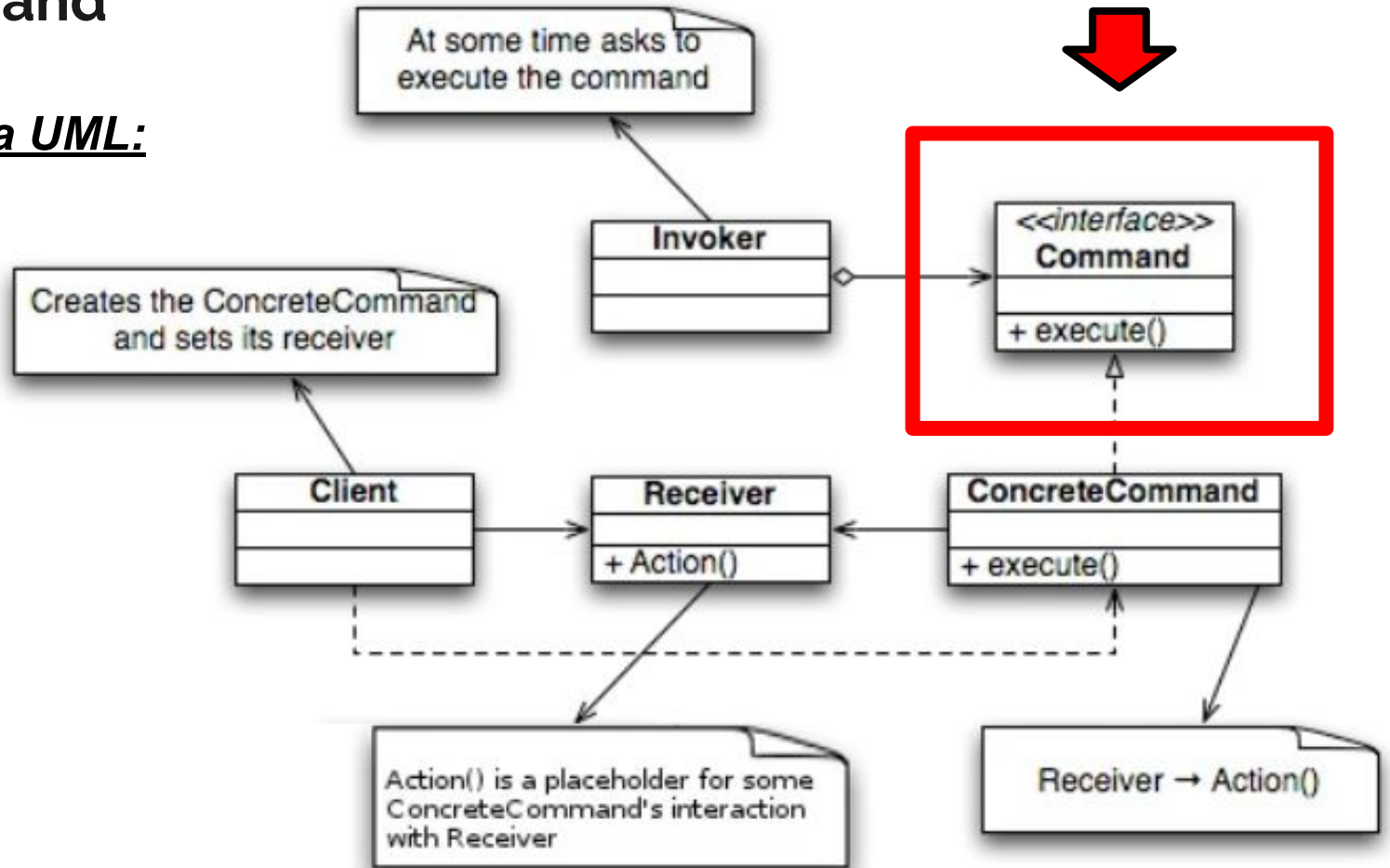
Command

- Diagrama UML:



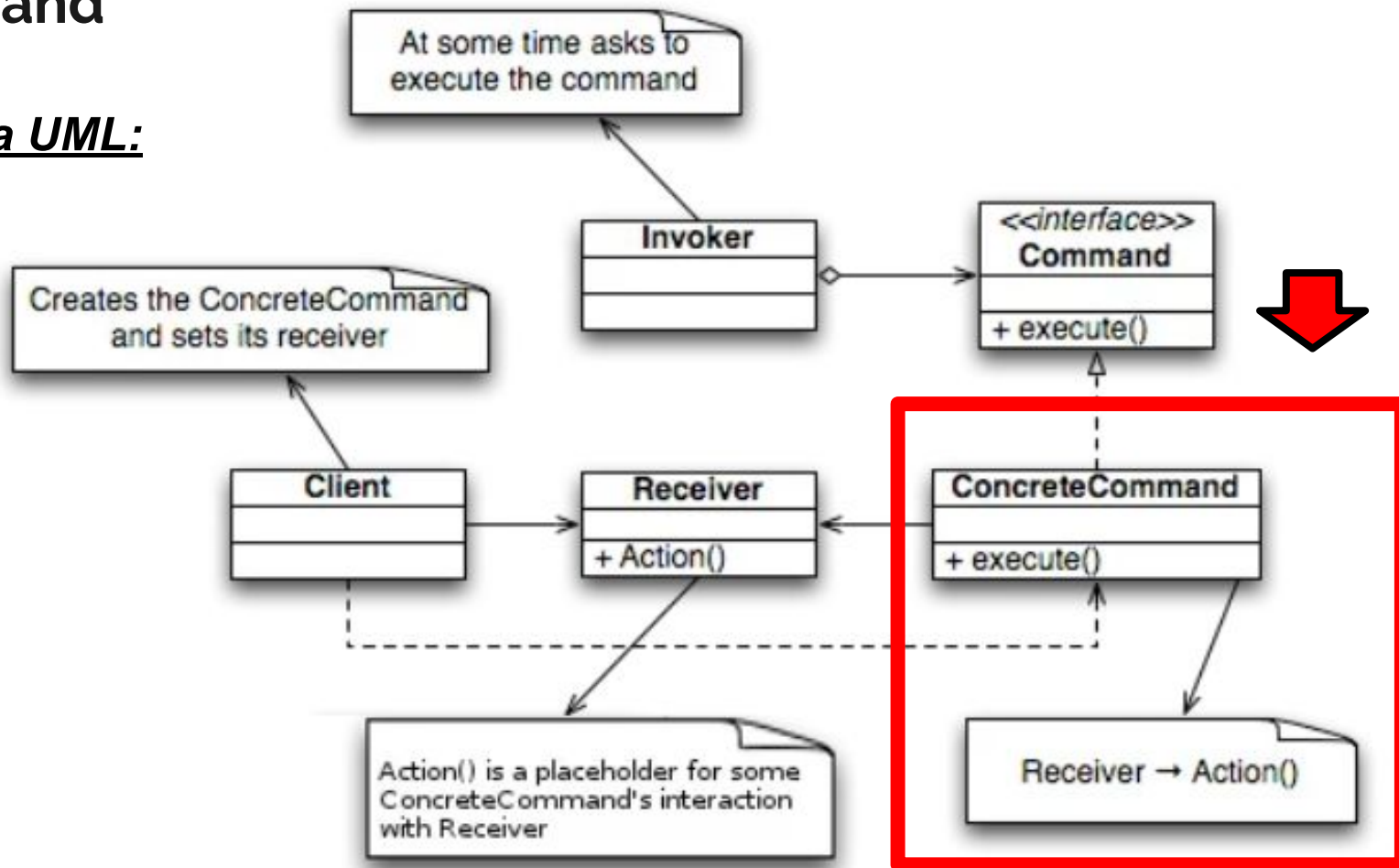
Command

- Diagrama UML:



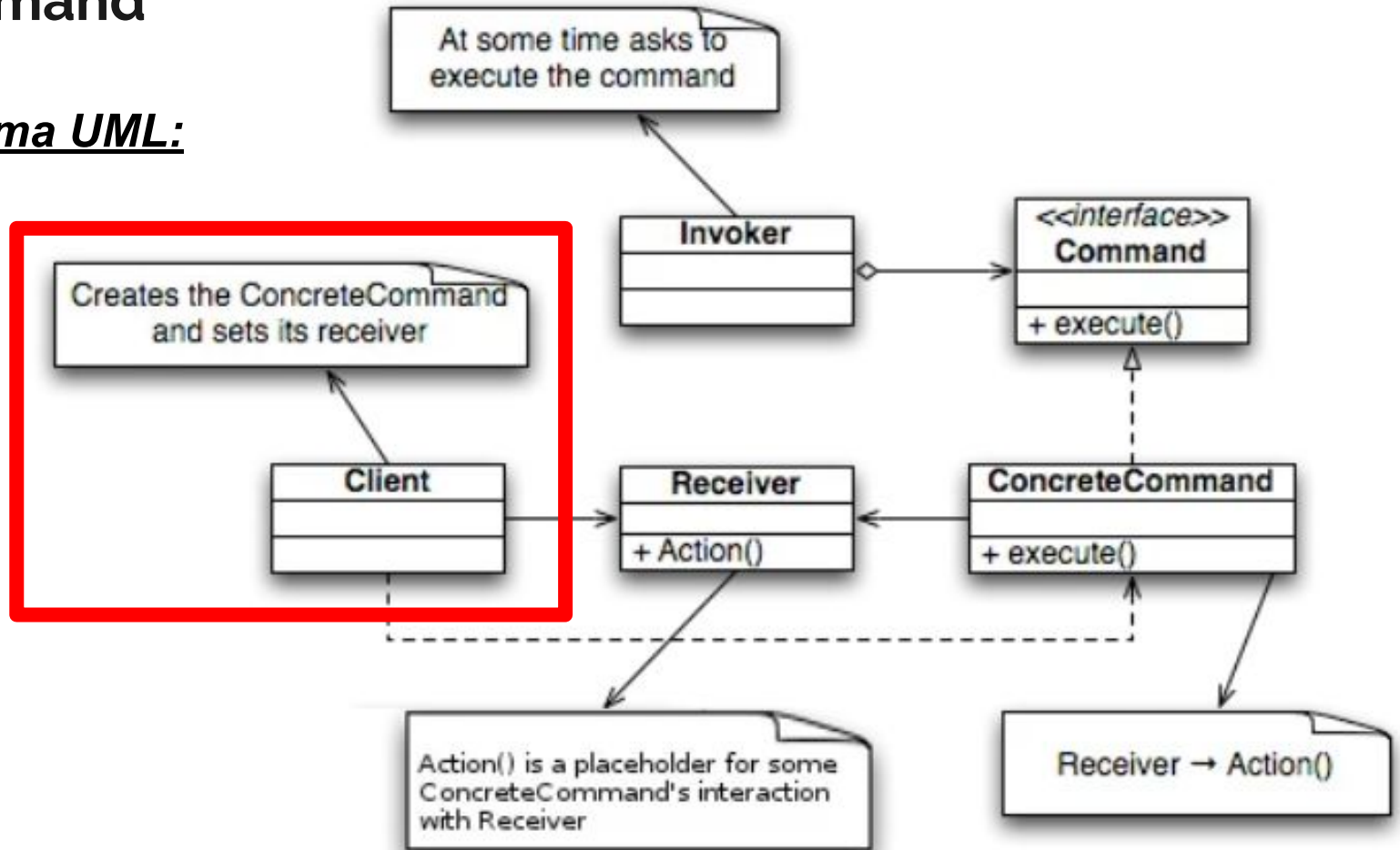
Command

- Diagrama UML:



Command

- Diagrama UML:



Command

```
1  from abc import ABC, abstractmethod
2
3  class Interface_Command(ABC):
4
5      @abstractmethod
6      def executar(self):
7          pass
8
9      @abstractmethod
10     def desfazer(self):
11         pass
```

Command

```
1  from Interface_Command import Interface_Command
2
3  class Invocador:
4
5      def __init__(self):
6          self.historico = []
7
8      def executar(self, comando: Interface_Command):
9          print("Executando tarefa ...")
10         comando.executar()
11         self.historico.append(comando)
12
13     def desfazer(self):
14         if self.historico:
15             ultimo = self.historico.pop()
16             ultimo.desfazer()
```

Command

```
1  class Receptor:
2
3      def backup(self):
4          print("Fazendo backup ...")
5
6      def enviar_email(self):
7          print("Enviando email ...")
8
9      def gerar_relatorio(self):
10         print("Gerando relatorio ...")
```

Command

```
1  from Interface_Command import Interface_Command
2  from Receptor import Receptor
3
4  class Backup_Command(Interface_Command):
5
6      def __init__(self, receptor: Receptor):
7          self.receptor = receptor
8
9      def executar(self):
10         self.receptor.backup()
11
12     def desfazer(self):
13         print("Desfazendo backup ...")
```

Command

```
1  from Interface_Command import Interface_Command
2  from Receptor import Receptor
3
4  class Email_Command(Interface_Command):
5
6      def __init__(self, receptor: Receptor):
7          self.receptor = receptor
8
9      def executar(self):
10         self.receptor.enviar_email()
11
12     def desfazer(self):
13         print("Desfazendo envio de email ...")
```

Command

```
1  from Interface_Command import Interface_Command
2  from Receptor import Receptor
3
4  class Relatorio_Command(Interface_Command):
5
6      def __init__(self, receptor: Receptor):
7          self.receptor = receptor
8
9      def executar(self):
10         self.receptor.gerar_relatorio()
11
12     def desfazer(self):
13         print("Desfazendo relatório ...")
```

Command

```
1  from Invocador import Invocador
2  from Receptor import Receptor
3  from Backup_Command import Backup_Command
4  from Email_Command import Email_Command
5  from Relatorio_Command import Relatorio_Command
6
7  receptor = Receptor()
8  invocador = Invocador()
9
10 invocador.executar(Backup_Command(receptor))
11 invocador.executar(Email_Command(receptor))
12 invocador.desfazer()
13 invocador.executar(Relatorio_Command(receptor))
```

Command

```
Executando tarefa ...  
Fazendo backup ...  
Executando tarefa ...  
Enviando email ...  
Desfazendo envio de email ...  
Executando tarefa ...  
Gerando relatorio ...
```

Command



- **Problema**: Vamos implementar um exemplo simples onde um comando controla o ligar e desligar de luzes usando o padrão Command. O objetivo é demonstrar como encapsular ações e executá-las de forma flexível.
 - A luz é ligada/desligada usando controle remoto
 - Não há necessidade de criar histórico de ações
 - Não há a necessidade de implementar a operação desfazer



Conteúdo da aula

- Introdução
- Classes em Python
- Introdução a Padrões de Projeto
 - Significado
 - Catálogo
- Conclusão
- Exercícios propostos
- Referências bibliográficas

Conclusão



Exercícios Propostos



- 1) Crie um programa que implemente corretamente o padrão de projeto Command.

Bibliografia Básica



- GAMMA, Erich; et al. **Padrões de projeto: soluções reutilizáveis de software orientado a objetos**. Porto Alegre: Bookman, 2008.
- FREEMAN, E.; FREEMAN, E. **Use a Cabeça! Padrões de Projetos (Design Patterns)**. 2a ed. Alta Books, 2007.
- METSKER, S. J. **Padrões de projeto em Java**. Porto Alegre: Bookman, 2004.

Bibliografia Complementar



- NEIL, T. **Padrões de design para aplicativos móveis**. São Paulo: Novatec, 2012.
- SHALLOWAY, A.; TROTT, J. R. **Explicando padrões de projeto – uma nova perspectiva em projeto orientado a objetos**. Porto Alegre: Bookman, 2004.

Material de Apoio



- MENEZES, Nilo Ney Coutinho. **Introdução a Programação com Python: Algoritmos e lógica de programação para iniciantes**. São Paulo: Novatec, 2019.
- <https://www.datacamp.com/pt/tutorial/encapsulation-in-python-object-oriented-programming>
- https://medium.com/@nelson.miranda_40644/implementa%C3%A7%C3%A3o-do-pad%C3%A3o-de-design-singleton-em-python-um-guia-para-iniciantes-c033dcd773e5
- <https://www.datacamp.com/pt/tutorial/python-abstract-classes>
- <https://www.geeksforgeeks.org/python/decorators-in-python/>

Material de Apoio



- <https://medium.com/@ankitlodh2002/exploring-pythons-abstraction-abstract-methods-decorators-and-magic-methods-demystified-2c6ec5e14dce>
- <https://www.devmedia.com.br/padrao-de-projeto-adapter-em-java/26467>
- <https://climaco.medium.com/design-pattern-adapter-231c90be41f2>
- <https://docs.python.org/3/glossary.html#term-type-hint>
- <https://www.datacamp.com/pt/tutorial/python-inheritance>
- <https://www.devmedia.com.br/padrao-de-projeto-template-method-em-java/26656>

Material de Apoio



- <https://medium.com/@jonesroberto/design-patterns-parte-24-template-method-69e3a7927dcd>
- <https://docs.python.org/pt-br/3/library/typing.html>
- <https://climaco.medium.com/design-pattern-composite-7fcc39c08ff2>
- <https://medium.com/@jonesroberto/desing-patterns-parte-10-composite-f7600cb3aad7>
- <https://medium.com/@jonesroberto/design-patterns-abstract-factory-da6b7057abce>
- <https://www.devmedia.com.br/padrao-abstract-factory/23030>

Material de Apoio



- <https://www.youtube.com/watch?v=ICbsumbkhnY>
- <https://medium.com/@jonesroberto/design-patterns-parte-19-mediator-a35af116eef1>
- https://medium.com/@nelson.miranda_40644/padr%C3%A3o-de-projeto-command-conceito-e-exemplos-em-python-18d8411814e1